

# Pod 的生命周期

## Pod 的生命周期

本页面讲述 Pod 的生命周期。Pod 遵循预定义的生命周期，起始于 Pending 阶段，如果至少其中有一个主要容器正常启动，则进入 Running，之后取决于 Pod 中是否有容器以失败状态结束而进入 Succeeded 或者 Failed 阶段。

和一个个独立的应用容器一样，Pod 也被认为是相对临时性（而不是长期存在）的实体。Pod 会被创建、赋予一个唯一的 ID（UID），并被调度到节点，并在终止（根据重启策略）或删除之前一直运行在该节点。如果一个死掉了，调度到该节点的 Pod 也被计划在给定超时期限结束后删除。

## Pod 生命期 {#pod-lifetime}

在 Pod 运行期间，kubelet 能够重启容器以处理一些失效场景。在 Pod 内部，Kubernetes 跟踪不同容器的状态并确定使 Pod 重新变得健康所需要采取的动作。

在 Kubernetes API 中，Pod 包含规约部分和实际状态部分。Pod 对象的状态包含了一组 Pod 状况（Conditions）。如果应用需要的话，你也可以向其中注入自定义的就绪态信息。

Pod 在其生命周期中只会被调度一次。将 Pod 分配到特定节点的过程称为**绑定**，而选择使用哪个节点的过程称为**调度**。一旦 Pod 被调度并绑定到某个节点，Kubernetes 会尝试在该节点上运行 Pod。Pod 会在该节点上运行，直到 Pod 停止或者被终止；如果 Kubernetes 无法在选定的节点上启动 Pod（例如，如果节点在 Pod 启动前崩溃），那么特定的 Pod 将永远不会启动。

你可以使用 Pod 调度就绪态来延迟 Pod 的调度，直到所有的**调度门控**都被移除。例如，你可能想要定义一组 Pod，但只有在所有 Pod 都被创建完成后才会触发调度。

## Pod 和故障恢复 {#pod-fault-recovery}

如果 Pod 中的某个容器失败，Kubernetes 可能会尝试重启特定的容器。有关细节参阅 Pod 如何处理容器问题。

然而，Pod 也可能以集群无法恢复的方式失败，在这种情况下，Kubernetes 不会进一步尝试修复 Pod；相反，Kubernetes 会删除 Pod 并依赖其他组件提供自动修复。

如果 Pod 被调度到某个节点之后失效，Pod 会被视为不健康，最终 Kubernetes 会删除 Pod。Pod 无法在因节点资源耗尽或者节点维护而被期间继续存活。

Kubernetes 使用一种高级抽象来管理这些相对而言可随时丢弃的 Pod 实例，称作。

任何给定的 Pod（由 UID 定义）从不会被“重新调度（rescheduled）”到不同的节点；相反，这一 Pod 可以被一个新的、几乎完全相同的 Pod 替换掉。如果你创建一个替换 Pod，它甚至可以拥有与旧 Pod 相同的名称（如 `.metadata.name`），但替换 Pod 将具有与旧 Pod 不同的 `.metadata.uid`。

Kubernetes 不保证现有 Pod 的替换 Pod 会被调度到与被替换的旧 Pod 相同的节点。

### 关联的生命期 {#associated-lifetimes}

如果某物声称其生命期与某 Pod 相同，例如存储，这就意味着该对象在此 Pod（UID 亦相同）存在期间也一直存在。如果 Pod 因为任何原因被删除，甚至某完全相同的替代 Pod 被创建时，这个相关的对象（例如这里的卷）也会被删除并重建。

### Pod 阶段 {#pod-phase}

Pod 的 `status` 字段是一个 `PodStatus` 对象，其中包含一个 `phase` 字段。

Pod 的阶段（Phase）是 Pod 在其生命周期中所处位置的简单宏观概述。该阶段并不是对容器或 Pod 状态的综合汇总，也不是为了成为完整的状态机。

Pod 阶段的数量和含义是严格定义的。除了本文档中列举的内容外，不应该再假定 Pod 有其他的 `phase` 值。

下面是 `phase` 可能的值：

- 取值：Pending（悬决）；描述：Pod 已被 Kubernetes 系统接受，但有一个或者多个容器尚未创建亦未运行。此阶段包括等待 Pod 被调度的时间和通过网络下载镜像的时间。
- 取值：Running（运行中）；描述：Pod 已经绑定到了某个节点，Pod 中所有的容器都已被创建。至少有一个容器仍在运行，或者正处于启动或重启状态。
- 取值：Succeeded（成功）；描述：Pod 中的所有容器都已成功结束，并且不会再重启。
- 取值：Failed（失败）；描述：Pod 中的所有容器都已终止，并且至少有一个容器是因为失败终止。也就是说，容器以非 0 状态退出或者被系统终止，且未被设置为自动重启。

- 取值：Unknown（未知）；描述：因为某些原因无法取得 Pod 的状态。这种情况通常是因为与 Pod 所在主机通信失败。

当 Pod 反复启动失败时，某些 kubectl 命令的 Status 字段中可能会出现 CrashLoopBackOff。同样，当 Pod 被删除时，某些 kubectl 命令的 Status 字段中可能会出现 Terminating。

确保不要将 **Status**（kubectl 用于用户直觉的显示字段）与 Pod 的 phase 混淆。Pod 阶段（phase）是 Kubernetes 数据模型和 Pod API 的一个明确的部分。

| NAMESPACE             | NAME            | READY | STATUS           | RESTARTS | AGE  |
|-----------------------|-----------------|-------|------------------|----------|------|
| alessandras-namespace | alessandras-pod | 0/1   | CrashLoopBackOff | 200      | 2d9h |

Pod 被赋予一个可以体面终止的期限，默认为 30 秒。你可以使用 --force 参数来强制终止 Pod。

从 Kubernetes 1.27 开始，除了静态 Pod 和没有 Finalizer 的强制终止 Pod 之外，kubelet 会将已删除的 Pod 转换到终止阶段（Failed 或 Succeeded 具体取决于 Pod 容器的退出状态），然后再从 API 服务器中删除。

如果某节点死掉或者与集群中其他节点失联，Kubernetes 会实施一种策略，将失去的节点上运行的所有 Pod 的 phase 设置为 Failed。

## 容器状态 {#container-states}

Kubernetes 会跟踪 Pod 中每个容器的状态，就像它跟踪 Pod 总体上的阶段一样。你可以使用容器生命周期回调 来在容器生命周期中的特定时间点触发事件。

一旦将 Pod 分派给某个节点，kubelet 就通过开始为 Pod 创建容器。容器的状态有三种：Waiting（等待）、Running（运行中）和 Terminated（已终止）。

要检查 Pod 中容器的状态，你可以使用 kubectl describe pod。其输出中包含 Pod 中每个容器的状态。

每种状态都有特定的含义：

### Waiting（等待） {#container-state-waiting}

如果容器并不处在 Running 或 Terminated 状态之一，它就处在 Waiting 状态。处于 Waiting 状态的容器仍在运行它完成启动所需要的操作：例如，从某个容器镜像仓库拉取容器镜像，或者向容器应用 数据等等。当你使用 kubectl 来查询包含 Waiting 状态的容器的 Pod 时，你也会看到一个 Reason 字段，其中给出了容器处于等待状态的原因。

## Running (运行中) {#container-state-running}

Running 状态表明容器正在执行状态并且没有问题发生。如果配置了 postStart 回调，那么该回调已经执行且已完成。如果你使用 kubectl 来查询包含 Running 状态的容器的 Pod 时，你也会看到关于容器进入 Running 状态的信息。

## Terminated (已终止) {#container-state-terminated}

处于 Terminated 状态的容器开始执行后，或者运行至正常结束或者因为某些原因失败。如果你使用 kubectl 来查询包含 Terminated 状态的容器的 Pod 时，你会看到容器进入此状态的原因、退出代码以及容器执行期间的起止时间。

如果容器配置了 preStop 回调，则该回调会在容器进入 Terminated 状态之前执行。

## Pod 如何处理容器问题 {#container-restarts}

Kubernetes 通过在 Pod spec 中定义的 restartPolicy 管理 Pod 内容器出现的失效。该策略决定了 Kubernetes 如何对由于错误或其他原因而退出的容器做出反应，其顺序如下：

1. **最初的崩溃：**Kubernetes 尝试根据 Pod 的 restartPolicy 立即重新启动。
2. **反复的崩溃：**在最初的崩溃之后，Kubernetes 对于后续重新启动的容器采用指数级回退延迟机制，如 restartPolicy 中所述。这一机制可以防止快速、重复的重新启动尝试导致系统过载。
3. **CrashLoopBackOff 状态：**这一状态表明，对于一个给定的、处于崩溃循环、反复失效并重启的容器，回退延迟机制目前正在生效。
4. **回退重置：**如果容器成功运行了一定时间（如 10 分钟），Kubernetes 会重置回退延迟机制，将新的崩溃视为第一次崩溃。

在实际部署中，CrashLoopBackOff 是在描述或列出 Pod 时从 kubectl 命令输出的一种状况或事件。当 Pod 中的容器无法正常启动，并反复进入尝试与失败的循环时就会出现。

换句话说，当容器进入崩溃循环时，Kubernetes 会应用容器重启策略中提到的指数级回退延迟机制。这种机制可以防止有问题的容器因不断进行启动失败尝试而导致系统不堪重负。

下列问题可以导致 CrashLoopBackOff：

- 应用程序错误导致的容器退出。
- 配置错误，如环境变量不正确或配置文件丢失。
- 资源限制，容器可能没有足够的内存或 CPU 正常启动。
- 如果应用程序没有在预期时间内启动服务，健康检查就会失败。

- 容器的存活探针或者启动探针返回失败结果，如探针部分所述。

要调查 CrashLoopBackOff 问题的根本原因，用户可以：

1. **检查日志：**使用 `kubectl logs` 检查容器的日志。这通常是诊断导致崩溃的问题的最直接方法。
2. **检查事件：**使用 `kubectl describe pod` 查看 Pod 的事件，这可以提供有关配置或资源问题的提示。
3. **审查配置：**确保 Pod 配置正确无误，包括环境变量和挂载卷，并且所有必需的外部资源都可用。
4. **检查资源限制：**确保容器被分配了足够的 CPU 和内存。有时，增加 Pod 定义中的资源可以解决问题。
5. **调试应用程序：**应用程序代码中可能存在错误或配置不当。在本地或开发环境中运行此容器镜像有助于诊断应用程序的特定问题。

## 容器重启 {#restart-policy}

当 Pod 中的某个容器停止或发生故障时，Kubernetes 可以重新启动此容器。但重启并不总是合适的；例如，只在 Pod 启动期间运行一次（如果成功）。你可以将重启配置为适用于所有 Pod 的策略，或者使用容器级别的配置（例如：在你定义或定义容器级别重载时）。

### 容器重启与弹性 {#container-restart-resilience}

Kubernetes 项目建议遵循云原生原则，包括能够应对未预告或随意重启的弹性设计。你可以通过让 Pod 失败并依赖自动替换，或者通过容器级别的弹性设计来实现。无论哪种方式，都有助于确保即使在部分故障的情况下，你的整体工作负载依然保持可用。

## Pod 级别容器重启策略

Pod 的 spec 中包含一个 `restartPolicy` 字段，其可能取值包括 `Always`、`OnFailure` 和 `Never`。默认值是 `Always`。

`restartPolicy` 应用于 Pod 中的和常规的 Init 容器。Sidecar 容器忽略 Pod 级别的 `restartPolicy` 字段：在 Kubernetes 中，Sidecar 被定义为 `initContainers` 内的一个条目，其容器级别的 `restartPolicy` 被设置为 `Always`。对于因错误而退出的 Init 容器，如果 Pod 级别 `restartPolicy` 为 `OnFailure` 或 `Always`，则 kubelet 会重新启动 Init 容器。

- `Always`：只要容器终止就自动重启容器。
- `OnFailure`：只有在容器错误退出（退出状态非零）时才重新启动容器。
- `Never`：不会自动重启已终止的容器。

## 重启行为比较

下表显示了在不同的重启策略和退出代码下容器的行为：

- 退出代码：0（成功）；restartPolicy: Always：重启；restartPolicy: OnFailure：不重启；restartPolicy: Never：不重启；边车容器：总是重启
- 退出代码：非零（失败）；restartPolicy: Always：重启；restartPolicy: OnFailure：重启；restartPolicy: Never：不重启；边车容器：总是重启

在选择 Deployment 还是 Job 时，重启行为尤为重要：

- **Deployment** 通常使用 restartPolicy: Always（唯一允许的值）来保持应用程序持续运行。
- **Job** 通常使用 restartPolicy: OnFailure 或 restartPolicy: Never 来妥善处理批处理作业。
- **边车容器**是初始化容器，无论 Pod 的 restartPolicy 设置如何，它们都会始终重启，因为它们拥有自己的容器级 restartPolicy: Always 设置。

## 示例场景

以下是一些具体示例，演示了不同的重启行为：

### 示例 1：Web 服务器，使用 restartPolicy: Always（通常用于 Deployment）

```
apiVersion: v1
kind: Pod
metadata:
  name: web-server
spec:
  restartPolicy: Always # 无论退出代码如何，容器都会重启
  containers:
  - name: nginx
    image: nginx:1.14.2
    # 如果此容器因任何原因崩溃或退出，它将被重新启动
```

### 示例 2：使用 restartPolicy: OnFailure 的批处理作业

```
apiVersion: batch/v1
kind: Job
metadata:
  name: data-processor
spec:
```



```
template:
spec:
  restartPolicy: OnFailure # 仅当退出代码非零时才重新启动
containers:
- name: processor
  image: busybox:1.28
  command: ['sh', '-c', 'echo "Processing data..."; exit 0']
  # 退出代码 0: 作业成功完成, 无需重启
  # 退出代码 1+: 容器重启以重试作业
```

### 示例 3: 使用 restartPolicy: Never 执行一次性作业

```
apiVersion: v1
kind: Pod
metadata:
  name: migration-task
spec:
  restartPolicy: Never # 无论退出代码如何, 都不要重新启动
containers:
- name: migrate
  image: busybox:1.28
  command: ['sh', '-c', 'echo "Running migration..."; exit 1']
  # 即使退出代码为 1 (失败), 容器也不会重启
  # Pod 将保持失败状态
```

### 边车容器和重启策略

边车容器具有与常规应用容器不同的特殊重启行为:

- **边车容器忽略 Pod 级别的 restartPolicy:** 它们使用自己的容器级 restartPolicy 字段, 该字段始终设置为 Always
- **独立生命周期:** 边车容器可以独立于主应用容器重启
- **持久运行:** 边车容器在 Pod 的整个生命周期内持续运行, 以提供支持服务

### 示例: 带有边车容器的 Pod

```
apiVersion: v1
kind: Pod
metadata:
  name: app-with-sidecar
spec:
```

```
restartPolicy: OnFailure # 仅适用于主容器
initContainers:
- name: logging-sidecar # 这是一个边车容器
  image: fluent/fluent-bit:1.8
  restartPolicy: Always # 无论退出代码如何，边车容器始终会重新启动。
  # 在 Pod 的整个生命周期内提供日志记录服务
containers:
- name: main-app # 这遵循 Pod 级别的重启策略
  image: nginx:1.14.2
  # 根据 Pod 的 OnFailure 策略，仅在失败（非零退出）时才会重启。
```

虽然主应用程序容器遵循 Pod 的 `restartPolicy: OnFailure`，但边车容器无论其退出代码如何都会重新启动，因为边车容器在容器级别会始终设置 `restartPolicy: Always`。

当 kubelet 根据配置的重启策略处理容器重启时，仅适用于同一 Pod 内替换容器并在同一节点上运行的重启。当 Pod 中的容器退出时，kubelet 会以指数级回退延迟机制（10 秒、20 秒、40 秒.....）重启容器，上限为 300 秒（5 分钟）。一旦容器顺利执行了 10 分钟，kubelet 就会重置该容器的重启延迟计时器。Sidecar 容器和 Pod 生命周期中解释了 `init containers` 在指定 `restartPolicy` 字段时的行为。

### 单个容器的重启策略与规则 {#container-restart-rules}

如果你的集群启用了 `ContainerRestartRules` 特性门控，你可以针对**单个容器**指定 `restartPolicy` 和 `restartPolicyRules` 来覆盖 Pod 重启策略。容器重启策略和规则适用于 Pod 中的以及常规的 Init 容器。

Kubernetes 原生的边车容器将其容器级别的 `restartPolicy` 设置为 `Always`。

容器重启会遵循与前文所述的 Pod 重启策略相同的指数回退机制。支持的容器重启策略有：

- `Always`：在任何原因的容器终止后都会自动重启容器。
- `OnFailure`：仅当容器因错误退出（非零退出状态）时才重启。
- `Never`：不自动重启已终止的容器。

此外，**单个容器**可以指定 `restartPolicyRules`。如果指定了 `restartPolicyRules` 字段，则**必须**同时指定容器的 `restartPolicy`。`restartPolicyRules` 定义了一系列在容器退出时应用的规则。每条规则由条件和动作组成。支持的条件是 `exitCodes`，用于将容器的退出码与给定值列表进行比较。支持的动作是 `Restart`，表示容器将被重启。这些规则会按顺序进行评估。一旦匹配成功，立即执行相应动作。如果没有任何规则的状况被匹配，Kubernetes 回退到容器配置的 `restartPolicy`。



例如，重启策略为 OnFailure 的某个 Pod 包含一个 try-once 容器。这样可以让 Pod 仅重启某些容器：

```
apiVersion: v1
kind: Pod
metadata:
  name: on-failure-pod
spec:
  restartPolicy: OnFailure
  containers:
  - name: try-once-container # 此容器只运行一次，因为 restartPolicy 设置为 Never。
    image: registry.k8s.io/busybox:1.27.2
    command: ['sh', '-c', 'echo "Only running once" && sleep 10 && exit 1']
    restartPolicy: Never
  - name: on-failure-container # 此容器将在失败时重启。
    image: registry.k8s.io/busybox:1.27.2
    command: ['sh', '-c', 'echo "Keep restarting" && sleep 1800 && exit 1']
```

下面是一个重启策略为 Always 的 Pod，其中包含一个只执行一次的 Init 容器。如果 Init 容器失败，则 Pod 也会失败。这样可以在初始化失败时让 Pod 失败，但在初始化成功后保持 Pod 运行：

```
apiVersion: v1
kind: Pod
metadata:
  name: fail-pod-if-init-fails
spec:
  restartPolicy: Always
  initContainers:
  - name: init-once # 这个 Init 容器只尝试一次。如果失败，Pod 将失败。
    image: registry.k8s.io/busybox:1.27.2
    command: ['sh', '-c', 'echo "Failing initialization" && sleep 10 && exit 1']
    restartPolicy: Never
  containers:
  - name: main-container # 一旦初始化成功，此容器会始终被重启。
    image: registry.k8s.io/busybox:1.27.2
    command: ['sh', '-c', 'sleep 1800 && exit 0']
```

下面是一个重启策略为 Never 的 Pod，其中包含的容器会在遇到特定的退出码时忽略之并重启。这种配置有助于区分可重启错误和不可重启错误：

```
apiVersion: v1
kind: Pod
metadata:
  name: restart-on-exit-codes
spec:
  restartPolicy: Never
  containers:
  - name: restart-on-exit-codes
    image: registry.k8s.io/busybox:1.27.2
    command: ['sh', '-c', 'sleep 60 && exit 0']
    restartPolicy: Never # 如果指定了规则，必须同时指定容器重启策略
    restartPolicyRules: # 仅当退出码为 42 时才重启容器
  - action: Restart
    exitCodes:
      operator: In
      values: [42]
```

重启规则可用于许多其他高级的生命周期管理场景。需要注意的是，重启规则会受到不一致性影响，这一点上与常规的重启策略相同。kubelet 重启、容器运行时垃圾收集、与控制平面的间歇性连接问题都可能导致状态丢失，容器可能会在你预期不应被重启的情况下被再次运行。

### *重启所有容器 {#restart-all-containers}*

如果你的集群已启用特性门控 `RestartAllContainersOnContainerExits`，你可以在容器级别的 `restartPolicyRules` 中指定 `RestartAllContainers` 作为一个动作。当容器的退出符合包含此动作的某个规则时，整个 Pod 被终止并就地重启。

这种“就地”重启比完全删除并重新创建 Pod 提供了更高效的方式来重置 Pod 的状态。这对于调度批处理作业或 AI/ML 训练任务这类开销高的工作负载尤其有价值。

### 就地重启 Pod 的工作原理

当触发 `RestartAllContainers` 动作时，kubelet 执行以下步骤：

1. **快速终止**：Pod 中所有正在运行的容器被终止。配置的 `terminationGracePeriodSeconds` 不会生效，配置的所有 `preStop` 回调也不会被执行。这样可以确保快速关闭容器。
2. **保留 Pod 资源**：Pod 的关键资源被保留：
  - Pod UID、IP 地址和网络命名空间

- Pod 沙箱及所挂接的所有设备
  - 所有卷，包括 emptyDir 和挂载卷
3. **更新 Pod 状态：**Pod 的状态被更新，同时 PodRestartInPlace 状况设置为 True，使重启过程可观测。
  4. **完整重启序列：**当所有容器终止后，将 PodRestartInPlace 状况设置为 False，然后 Pod 开始标准的启动流程：
    - **Init 容器按顺序重新运行。**
    - 边车和普通容器启动。

此特性的一个关键点是**所有**容器都会被重启，包括之前已成功完成或失败的容器。RestartAllContainers 动作会重载所有已配置的容器级或 Pod 级别的 restartPolicy。

这种机制在需要为所有容器提供干净环境的场景中非常有用，例如：

- 当 Init 容器设置的环境可能被破坏时，此特性确保重新执行此设置过程。
- 边车容器可以监控主应用的健康状态，如果该应用进入不可恢复的状态，则触发整个 Pod 重启。

考虑一种工作负载，其中 watcher 边车负责在主应用出错时从已知良好状态重启主应用。watcher 可以以特定代码退出，从而触发 worker Pod 的就地完整重启。

在此示例中：

- Pod 的整体 restartPolicy 为 Never。
- watcher-sidecar 执行命令后以退出码 88 退出。
- 退出码符合规则，触发 RestartAllContainers 动作。
- 整个 Pod（包括 setup-environment init 容器和 main-application 容器）随后就地重启。Pod 保留其 UID、沙箱、IP 和卷。

### 减少容器重启延迟 {#Reduced-container-restart-delay}

启用 Alpha 特性开关 ReduceDefaultCrashLoopBackOffDecay 后，集群中容器启动重试的初始延迟将从 10 秒减少到 1 秒，之后每次重启延迟时间按 2 倍指数增长，直到达到最大延迟 60 秒（之前为 300 秒，即 5 分钟）。

如果你同时使用了下面介绍的 Alpha 特性 KubeletCrashLoopBackOffMax，那么单个节点上可能会有不同的最大延迟值。

## 可配置的容器重启延迟 {#configurable-container-restart-delay}

启用特性门控 KubeletCrashLoopBackOffMax 后，你可以重新配置容器启动重试之间的最大延迟，默认值为 300 秒（5 分钟）。此配置是针对每个节点使用 kubelet 配置进行设置的。在你的 kubelet 配置中，在 crashLoopBackOff 下设置 maxContainerRestartPeriod 字段，取值范围在 "1s" 到 "300s" 之间。如上文容器重启策略所述，该节点上的延迟仍将从 10 秒开始，并在每次重启后以指数方式增加 2 倍，但现在其上限将被限制为你所配置的最大值。如果你配置的 maxContainerRestartPeriod 小于默认初始值 10 秒，则初始延迟将被设置为配置的最大值。

参见以下 kubelet 配置示例：

```
# 容器重启延迟将从 10 秒开始，每次重启增加 2 倍  
# 最高达到 100 秒
```

```
kind: KubeletConfiguration  
crashLoopBackOff:  
  maxContainerRestartPeriod: "100s"
```

```
# 容器重启之间的延迟将始终为 2 秒
```

```
kind: KubeletConfiguration  
crashLoopBackOff:  
  maxContainerRestartPeriod: "2s"
```

如果你将此特性与上文提到的 Alpha 特性 ReduceDefaultCrashLoopBackOffDecay 一起使用，那么集群的初始退避时间和最大退避时间默认值将不再是 10 秒和 300 秒，而是 1 秒和 60 秒。每个节点上的配置优先于 ReduceDefaultCrashLoopBackOffDecay 所设置的默认值，即使这会导致某些节点的最大退避时间比集群中的其他节点更长。

## Pod 状况 {#pod-conditions}

Pod 有一个 PodStatus 对象，其中包含一个 PodConditions 数组。Pod 可能通过也可能未通过其中的一些状况测试。Kubelet 管理以下 PodCondition：

- PodScheduled：Pod 已经被调度到某节点；
- PodReadyToStartContainers：Pod 沙箱被成功创建并且配置了网络（Beta 特性，默认启用）；
- ContainersReady：Pod 中所有容器都已就绪；
- Initialized：所有的 Init 容器都已成功完成；
- Ready：Pod 可以为请求提供服务，并且应该被添加到对应服务的负载均衡池中。
- DisruptionTarget：由于干扰（例如抢占、驱逐或垃圾回收），Pod 即将被终止。

- PodResizePending: 已请求对 Pod 进行调整大小, 但尚无法应用。详见 Pod 调整大小状态。
- PodResizeInProgress: Pod 正在调整大小中。详见 Pod 调整大小状态。
- 字段名称: type; 描述: Pod 状况的名称
- 字段名称: status; 描述: 表明该状况是否适用, 可能的取值有 "True"、"False" 或 "Unknown"
- 字段名称: lastProbeTime; 描述: 上次探测 Pod 状况时的时间戳
- 字段名称: lastTransitionTime; 描述: Pod 上次从一种状态转换到另一种状态时的时间戳
- 字段名称: reason; 描述: 机器可读的、驼峰编码 (UpperCamelCase) 的文字, 表述上次状况变化的原因
- 字段名称: message; 描述: 人类可读的消息, 给出上次状态转换的详细信息

## Pod 就绪态 {#pod-readiness-gate}

你的应用可以向 PodStatus 中注入额外的反馈或者信号: **Pod Readiness (Pod 就绪态)**。要使用这一特性, 可以设置 Pod 规约中的 readinessGates 列表, 为 kubelet 提供一组额外的状况供其评估 Pod 就绪态时使用。

就绪态门控基于 Pod 的 status.conditions 字段的当前值来做决定。如果 Kubernetes 无法在 status.conditions 字段中找到某状况, 则该状况的状态值默认为 "False"。

这里是一个例子:

```
kind: Pod
...
spec:
  readinessGates:
    - conditionType: "www.example.com/feature-1"
status:
  conditions:
    - type: Ready                # 内置的 Pod 状况
      status: "False"
      lastProbeTime: null
      lastTransitionTime: 2018-01-01T00:00:00Z
    - type: "www.example.com/feature-1"    # 额外的 Pod 状况
      status: "False"
      lastProbeTime: null
      lastTransitionTime: 2018-01-01T00:00:00Z
  containerStatuses:
```

```
- containerID: docker://abcd...  
  ready: true  
...
```

你所添加的 Pod 状况名称必须满足 Kubernetes 标签键名格式。

### Pod 就绪态的状态 {#pod-readiness-status}

命令 `kubectl patch` 不支持修改对象的状态。如果需要设置 Pod 的 `status.conditions`，应用或者

需要使用 PATCH 操作。你可以使用 Kubernetes 客户端库之一来编写代码，针对 Pod 就绪态设置定制的 Pod 状况。

对于使用定制状况的 Pod 而言，只有当下面的陈述都适用时，该 Pod 才会被评估为就绪：

- Pod 中所有容器都已就绪；
- `readinessGates` 中的所有状况都为 True 值。

当 Pod 的容器都已就绪，但至少一个定制状况没有取值或者取值为 False，kubelet 将 Pod 的状况设置为 `ContainersReady`。

### Pod 网络就绪 {#pod-has-network}

在其早期开发过程中，这种状况被命名为 `PodHasNetwork`。

在 Pod 被调度到某节点后，它需要被 kubelet 接受并且挂载所需的存储卷。一旦这些阶段完成，Kubelet 将与容器运行时（使用）一起为 Pod 生成运行时沙箱并配置网络。如果启用了 `PodReadyToStartContainersCondition` 特性门控（Kubernetes 版本中默认启用），`PodReadyToStartContainers` 状况会被添加到 Pod 的 `status.conditions` 字段中。

当 kubelet 检测到 Pod 不具备配置了网络的运行时沙箱时，`PodReadyToStartContainers` 状况将被设置为 False。以下场景中将会发生这种状况：

- 在 Pod 生命周期的早期阶段，kubelet 还没有开始使用容器运行时为 Pod 设置沙箱时。
- 在 Pod 生命周期的末期阶段，Pod 的沙箱由于以下原因被销毁时：
  - 节点重启时 Pod 没有被驱逐
  - 对于使用虚拟机进行隔离的容器运行时，Pod 沙箱虚拟机重启时，需要创建一个新的沙箱和全新的容器网络配置。



在运行时插件成功完成 Pod 的沙箱创建和网络配置后，kubelet 会将 PodReadyToStartContainers 状况设置为 True。当 PodReadyToStartContainers 状况设置为 True 后，Kubelet 可以开始拉取容器镜像和创建容器。

对于带有 Init 容器的 Pod，kubelet 会在 Init 容器成功完成后将 Initialized 状况设置为 True（这发生在运行时成功创建沙箱和配置网络之后），对于没有 Init 容器的 Pod，kubelet 会在创建沙箱和网络配置开始之前将 Initialized 状况设置为 True。

## 调整 Pod 大小 {#pod-resize}

Kubernetes 支持在 Pod 创建后更改分配给 Pod 的 CPU 和内存资源。（对于其他基础设施资源，你需要使用特定于这些资源的不同技术。）调整 CPU 和内存资源主要有两种方法：

### 原地 Pod 调整大小 {#pod-resize-inplace}

你可以调整 Pod 的容器级别 CPU 和内存资源，而无需重建 Pod。这亦被称为**原地 Pod 垂直扩缩**。这允许你在可能避免应用程序中断的同时，调整运行容器的资源配置。

要执行原地调整大小，你可以使用 /resize 子资源更新 Pod 的期望状态。然后，kubelet 会尝试将新的资源值应用到运行中的容器。Pod PodResizePending 和 PodResizeInProgress（在 Pod 状况中描述）指示调整大小操作的状态。有关调整大小状态的更多详情，请参见 容器调整大小状态。

就地调整大小的关键考量：

- 仅 CPU 和内存资源可以原地调整大小。
- Pod 的服务质量（QoS）类在创建时确定，且不能通过调整大小来更改。
- 你可以使用容器规约中的 resizePolicy 配置是否需要重启容器以进行调整大小。

有关执行原地调整大小的详细说明，请参见调整分配给容器的 CPU 和内存资源。

### 通过启动替代 Pod 进行调整大小

更改 Pod 资源更云原生的方法是通过管理它的工作负载资源（如 Deployment 或 StatefulSet）。当你更新 Pod 模板中的资源规约时，工作负载的控制器会根据其更新策略创建具有更新资源的新 Pod 并终止旧 Pod。

这种方法：

- 适用于任何 Kubernetes 版本。
- 可以更改任何 Pod 规约，而不仅仅是资源。

- 会导致 Pod 替换，因此你应该设计你的工作负载来处理计划内的干扰。考虑使用 PodDisruptionBudget 来控制可用性。
- 要求你的 Pod 由工作负载资源管理。

你也可以使用 VerticalPodAutoscaler 来自动管理 Pod 资源建议和更新。

## 容器探针 {#container-probes}

**probe** 是由 kubelet 对容器执行的定期诊断。要执行诊断，kubelet 既可以在容器内执行代码，也可以发出一个网络请求。

### 检查机制 {#probe-check-methods}

使用探针来检查容器有四种不同的方法。每个探针都必须准确定义为这四种机制中的一种：

**exec** : 在容器内执行指定命令。如果命令退出时返回码为 0 则认为诊断成功。

**grpc** : 使用 gRPC 执行一个远程过程调用。目标应该实现 gRPC 健康检查。如果响应的状态是 "SERVING"，则认为诊断成功。

**httpGet** : 对容器的 IP 地址上指定端口和路径执行 HTTP GET 请求。如果响应的状态码大于等于 200 且小于 400，则诊断被认为是成功的。有关 kubelet 如何跟踪重定向的更多信息，请参阅配置探测。

**tcpSocket** : 对容器的 IP 地址上的指定端口执行 TCP 检查。如果端口打开，则诊断被认为是成功的。如果远程系统（容器）在打开连接后立即将其关闭，这算作是健康的。

和其他机制不同，exec 探针的实现涉及每次执行时创建/复制多个进程。因此，在集群中具有较高 pod 密度、较低的 initialDelaySeconds 和 periodSeconds 时长的时候，配置任何使用 exec 机制的探针可能会增加节点的 CPU 负载。这种场景下，请考虑使用其他探针机制以避免额外的开销。

### 探测结果 {#probe-outcome}

每次探测都将获得以下三种结果之一：

**Success（成功）** : 容器通过了诊断。

**Failure（失败）** : 容器未通过诊断。

**Unknown（未知）** : 诊断失败，因此不会采取任何行动。

## 探测类型 {#types-of-probe}

针对运行中的容器，kubelet 可以选择是否执行以下三种探针，以及如何针对探测结果作出反应：

**livenessProbe** : 指示容器是否正在运行。如果存活态探测失败，则 kubelet 会杀死容器，并且容器将根据其重启策略决定未来。如果容器不提供存活探针，则默认状态为 Success。

**readinessProbe** : 指示容器是否准备好为请求提供服务。如果就绪态探测失败，

控制器将从与该 Pod 匹配的所有 Service 的 EndpointSlice 中删除该 Pod 的 IP 地址。初始延迟之前的就绪态的状态值默认为 Failure。如果容器不提供就绪态探针，则默认状态为 Success。

**startupProbe** : 指示容器中的应用是否已经启动。如果提供了启动探针，则所有其他探针都会被禁用，直到此探针成功为止。如果启动探测失败，kubelet 将杀死容器，而容器依其重启策略进行重启。如果容器没有提供启动探测，则默认状态为 Success。

如欲了解如何设置存活态、就绪态和启动探针的进一步细节，可以参阅配置存活态、就绪态和启动探针。

### 何时该使用存活态探针? {#when-should-you-use-a-liveness-probe}

如果容器中的进程能够在遇到问题或不健康的情况下自行崩溃，则不一定需要存活态探针； kubelet 将根据 Pod 的 restartPolicy 自动执行修复操作。

如果你希望容器在探测失败时被杀死并重新启动，那么请指定一个存活态探针，并指定 restartPolicy 为 "Always" 或 "OnFailure"。

### 何时该使用就绪态探针? {#when-should-you-use-a-readiness-probe}

如果要仅在探测成功时才开始向 Pod 发送请求流量，请指定就绪态探针。在这种情况下，就绪态探针可能与存活态探针相同，但是规约中的就绪态探针的存在意味着 Pod 将在启动阶段不接收任何数据，并且只有在探针探测成功后才开始接收数据。

如果你希望容器能够自行进入维护状态，也可以指定一个就绪态探针，检查某个特定于就绪态的因此不同于存活态探测的端点。

如果你的应用程序对后端服务有严格的依赖性，你可以同时实现存活态和就绪态探针。当应用程序本身是健康的，存活态探针检测通过后，就绪态探针会额外检查每个所需的后端服务是否可用。这可以帮助你避免将流量导向只能返回错误信息的 Pod。

如果你的容器需要在启动期间加载大型数据、配置文件或执行迁移，你可以使用启动探针。然而，如果你想区分已经失败的应用和仍在处理其启动数据的应用，你可能更倾向于使用就绪探针。

请注意，如果你只是想在 Pod 被删除时能够排空请求，则不一定需要使用就绪态探针；当 Pod 被删除时，EndpointSlice 中对应的端点会更新其状况：该端点的 ready 状况将被设置为 false，因此负载均衡器不会再将该 Pod 用于常规流量。关于 kubelet 如何处理 Pod 删除的更多信息，请参见 Pod 终止。

### 何时该使用启动探针？{#when-should-you-use-a-startup-probe}

对于所包含的容器需要较长时间才能启动就绪的 Pod 而言，启动探针是有用的。你不再需要配置一个较长的存活态探测时间间隔，只需要设置另一个独立的配置选定，对启动期间的容器执行探测，从而允许使用远远超出存活态时间间隔所允许的时长。

如果你的容器启动时间通常超出  $(\text{initialDelaySeconds} + \text{failureThreshold} \times \text{periodSeconds})$  总值，你应该设置一个启动探测，对存活态探针所使用的同一端点执行检查。periodSeconds 的默认值是 10 秒。你应该将其 failureThreshold 设置得足够高，以便容器有充足的时间完成启动，并且避免更改存活态探针所使用的默认值。这一设置有助于减少死锁状况的发生。

## Pod 的终止 {#pod-termination}

由于 Pod 所代表的是在集群中节点上运行的进程，当不再需要这些进程时允许其体面地终止是很重要的。一般不应武断地使用 KILL 信号终止它们，导致这些进程没有机会完成清理操作。

设计的目标是令你能够请求删除进程，并且知道进程何时被终止，同时也能够确保删除操作终将完成。当你请求删除某个 Pod 时，集群会记录并跟踪 Pod 的体面终止周期，而不是直接强制地杀死 Pod。在存在强制关闭设施的前提下，会尝试体面地终止 Pod。

通常 Pod 体面终止的过程为：kubelet 先发送一个带有体面超时限期的 TERM（又名 SIGTERM）信号到每个容器中的主进程，将请求发送到容器运行时来尝试停止 Pod 中的容器。停止容器的这些请求由容器运行时以异步方式处理。这些请求的处理顺序无法被保证。许多容器运行时遵循容器镜像内定义的 STOPSIGNAL 值，如果不同，则发送容器镜像中配置的 STOPSIGNAL，而不是 TERM 信号。一旦超出了体面终止限期，容器运行时向所有剩余进程发送 KILL 信号，之后 Pod 就会被从集群中移除。如果 kubelet 或者容器运行时的管理服务在等待进程终止期间被重启，集群会从头开始重试，赋予 Pod 完整的体面终止限期。

## 终止信号 {#pod-termination-stop-signals}

用于终止容器的终止信号可以通过容器镜像中的 STOPSIGNAL 指令进行定义。如果镜像中未定义终止信号，容器运行时（containerd 和 CRI-O 都是 SIGTERM）会使用默认的终止信号来终止容器。

## 定义自定义终止信号 {#Defining-custom-stop-signals}

如果启用了 ContainerStopSignals 特性门控（feature gate），你可以通过容器的生命周期（Lifecycle）配置自定义的终止信号。在容器生命周期中定义终止信号时，Pod 的 spec.os.name 字段必须存在。可用的信号列表取决于 Pod 调度到的操作系统。对于调度到 Windows 节点的 Pod，仅支持 SIGTERM 和 SIGKILL 信号。

以下是一个定义了自定义终止信号的 Pod 示例：

```
spec:
  os:
    name: linux
  containers:
  - name: my-container
    image: container-image:latest
    lifecycle:
      stopSignal: SIGUSR1
```

如果在生命周期中定义了终止信号，则会覆盖容器镜像中定义的信号。如果容器规约中未定义终止信号，则容器将回退到默认行为。

## Pod 终止流程 {#pod-termination-flow}

Pod 终止流程，如下例所示：

1. 你使用 kubectl 工具手动删除某个特定的 Pod，而该 Pod 的体面终止限期是默认值（30 秒）。
2. API 服务器中的 Pod 对象被更新，记录涵盖体面终止限期在内 Pod 的最终死期，超出所计算时间点则认为 Pod 已死（dead）。如果你使用 kubectl describe 来查验你正在删除的 Pod，该 Pod 会显示为 "Terminating"（正在终止）。在 Pod 运行所在的节点上：kubelet 一旦看到 Pod 被标记为正在终止（已经设置了体面终止限期），kubelet 即开始本地的 Pod 关闭过程。

1. 如果 Pod 中的容器之一定义了 preStop 回调 且 Pod 规约中的 terminationGracePeriodSeconds 未设为 0， kubelet 开始在容器内运行该回调逻辑。默认的 terminationGracePeriodSeconds 设置为 30 秒。

如果 preStop 回调在体面期结束后仍在运行， kubelet 将请求短暂的、一次性的体面期延长 2 秒。

如果 preStop 回调所需要的时间长于默认的体面终止限期，你必须修改 terminationGracePeriodSeconds 属性值来使其正常工作。

2. kubelet 接下来触发容器运行时发送 TERM 信号给每个容器中的进程 1。

如果 Pod 中定义了， 则存在特殊排序。否则， Pod 中的容器会在不同的时间和任意的顺序接收 TERM 信号。如果关闭顺序很重要，考虑使用 preStop 钩子进行同步（或者切换为使用 Sidecar 容器）。

3. 在 kubelet 启动 Pod 的体面关闭逻辑的同时，控制平面会评估是否将关闭的 Pod 从对应的 EndpointSlice 对象中移除，过滤条件是 Pod 被对应的以某 选定。

和其他工作负载资源不再将关闭进程中的 Pod 视为合法的、能够提供服务的副本。

关闭动作很慢的 Pod 不应继续处理常规服务请求，而应开始终止并完成对打开的连接的处理。一些应用程序不仅需要完成对打开的连接的处理，还需要更进一步的体面终止逻辑 - 比如：排空和完成会话。

任何正在终止的 Pod 所对应的端点都不会立即从 EndpointSlice 中被删除， EndpointSlice API 会公开一个状态来指示其处于 终止状态。正在终止的端点始终将其 ready 状态设置为 false（为了向后兼容 1.26 之前的版本）， 因此负载均衡器不会将其用于常规流量。

如果需要排空正被终止的 Pod 上的流量，可以将 serving 状况作为实际的就绪状态。你可以在教程 探索 Pod 及其端点的终止行为 中找到有关如何实现连接排空的更多详细信息。

4. kubelet 确保 Pod 被关闭和终止

1. 超出终止宽限期限时，如果 Pod 中仍有容器在运行， kubelet 会触发强制关闭过程。容器运行时向 Pod 中所有容器内仍在运行的进程发送 SIGKILL 信号。 kubelet 也会清理隐藏的 pause 容器，如果容器运行时使用了这种容器的话。



2. kubelet 将 Pod 转换到终止阶段（Failed 或 Succeeded，具体取决于其容器的结束状态）。
3. kubelet 通过将宽限期设置为 0（立即删除），触发从 API 服务器强制移除 Pod 对象的操作。
4. API 服务器删除 Pod 的 API 对象，从任何客户端都无法再看到该对象。

### 强制终止 Pod {#pod-termination-forced}

对于某些工作负载及其 Pod 而言，强制删除很可能会带来某种破坏。

默认情况下，所有的删除操作都会附有 30 秒钟的宽限期限。kubectl delete 命令支持 --grace-period= 选项，允许你重载默认值，设定自己希望的期限值。

将宽限期限强制设置为 0 意味着立即从 API 服务器删除 Pod。如果 Pod 仍然运行于某节点上，强制删除操作会触发 kubelet 立即执行清理操作。

使用 kubectl 时，你必须在设置 --grace-period=0 的同时额外设置 --force 参数才能发起强制删除请求。

执行强制删除操作时，API 服务器不再等待来自 kubelet 的、关于 Pod 已经在原来运行的节点上终止执行的确认消息。API 服务器直接删除 Pod 对象，这样新的与之同名的 Pod 即可以被创建。在节点侧，被设置为立即终止的 Pod 仍然会在被强行杀死之前获得一点点的宽限时间。

马上删除时不等待确认正在运行的资源已被终止。这些资源可能会无限期地继续在集群上运行。

如果你需要强制删除 StatefulSet 的 Pod，请参阅从 StatefulSet 中删除 Pod 的任务文档。

### Pod 关闭和 Sidecar 容器 {#termination-with-sidecars}

如果你的 Pod 包含一个或多个 Sidecar 容器（重启策略为 Always 的 Init 容器），kubelet 将延迟向这些 Sidecar 容器发送 TERM 信号，直到最后一个主容器已完全终止。Sidecar 容器将按照它们在 Pod 规约中被定义的相反顺序被终止。这样确保了 Sidecar 容器继续为 Pod 中的其他容器提供服务，直到完全不再需要为止。

这意味着主容器的慢终止也会延迟 Sidecar 容器的终止。如果在终止过程完成之前宽限期已到，Pod 可能会进入强制终止阶段。在这种情况下，Pod 中所有剩余的容器将在某个短宽限期内被同时终止。

同样地，如果 Pod 有一个 preStop 钩子超过了终止宽限期，可能会发生紧急终止。总体而言，如果你以前使用 preStop 钩子来控制没有 Sidecar 的 Pod 中容器的终止顺序，你现在可以移除这些钩子，允许 kubelet 自动管理 Sidecar 的终止。

## Pod 的垃圾收集 {#pod-garbage-collection}

对于已失败的 Pod 而言，对应的 API 对象仍然会保留在集群的 API 服务器上，直到用户或者进程显式地将其删除。

Pod 的垃圾收集器（PodGC）是控制平面的控制器，它会在 Pod 个数超出所配置的阈值（根据 kube-controller-manager 的 terminated-pod-gc-threshold 设置）时删除已终止的 Pod（阶段值为 Succeeded 或 Failed）。这一行为会避免随着时间演进不断创建和终止 Pod 而引起的资源泄露问题。

此外，PodGC 会清理满足以下任一条件的所有 Pod：

1. 孤儿 Pod - 绑定到不再存在的节点，
2. 计划外终止的 Pod
3. 终止过程中的 Pod，绑定到有 node.kubernetes.io/out-of-service 污点的未就绪节点。

在清理 Pod 的同时，如果它们处于非终止状态阶段，PodGC 也会将它们标记为失败。此外，PodGC 在清理孤儿 Pod 时会添加 Pod 干扰状况。参阅 Pod 干扰状况 了解更多详情。

## kubelet 重启期间的 Pod 行为 {#kubelet-restarts}

如果你重启 kubelet，Pod（及其容器）即使在重启过程中也会继续运行。如果某个节点上存在正在运行的 Pod，停止或重启该节点上的 kubelet **不会**在 kubelet 自身停止之前让 kubelet 停止所有本地 Pod。如果需要停止节点上的 Pod，可以使用 kubectl drain。

## 检测 kubelet 重启 {#detection-of-kubelet-restarts}

当 kubelet 启动时，它会检查是否已经存在一个绑定了 Pod 的 Node。如果该 Node 的 Ready 状况保持不变，也就是说该状况没有从 true 变为 false，Kubernetes 就会将其检测为一次 **kubelet 重启**。（也可能通过其他方式重启 kubelet，例如为修复某个节点缺陷而重启；在这些情况下，Kubernetes 会选择更安全的处理方式，例如先停止再启动 kubelet。）

当 kubelet 重启时，容器状态将基于特性门控的设置以不同方式进行管理：

- 默认情况下，kubelet 在重启后不更改容器状态。之前设置为 ready: true 状态的容器仍然保持就绪。

如果你停止 kubelet 的时间足够长，导致其连续多次节点心跳检查失败，并且在重启 kubelet 之前又等待了一段时间，Kubernetes 可能会开始从该节点驱逐 Pod。但是，即使开始发生 Pod 驱逐，Kubernetes 也不会将这些 Pod 中的单个容器标记为 ready: false。Pod 级别的驱逐是在控制平面因心跳失败而将节点标记为 node.kubernetes.io/not-ready 之后发生的。

- 在 Kubernetes 中，你可以选择启用一种**传统的行为**：在 kubelet 重启后，总是将容器的 ready 状态修改为 false。

这种传统行为在很长一段时间内都是默认设置的，但给 Kubernetes 用户带来了一些问题，尤其是在大规模部署场景中。虽然此特性门控允许暂时回退到这种传统行为，但 Kubernetes 项目建议如果你遇到相关问题，应提交 Bug 报告。

ChangeContainerStatusOnKubeletRestart 特性门控 将在未来被移除。

- 动手实践为容器生命周期时间关联处理程序。
- 动手实践配置存活态、就绪态和启动探针。
- 进一步了解容器生命周期回调。
- 进一步了解 Sidecar 容器。
- 关于 API 中定义的有关 Pod 和容器状态的详细规范信息，可参阅 API 参考文档中 Pod 的 status 字段。